

10.1. Acceso a datos desde fichero

PHP, al igual que otros lenguajes de programación, dispone de una gama de funciones de acceso a ficheros.

Veremos las básicas:

- abrir (fopen)
- cerrar (fclose)
- leer (fgets)
- escribir (fputs).

fopen (archivo, modo)

Con esta función abrimos un fichero, bien sea local o una dirección de internet (http:// o ftp://).

La función **fopen** nos devuelve un puntero al fichero (indicador de archivo) que nos servirá para hacer referencia al archivo abierto. Si no puede abrirlo nos retorna false. Si no encuentra el archivo se produce error (para evitar que se muestre @fopen).

Con fopen podemos abrir un archivo de los siguientes modos:

- **r** solo lectura
- **r+** lectura y escritura
- **w** solo escritura. Si no existe el archivo lo crea, si ya existe lo machaca.
- **w+** lectura y escritura. Si no existe el archivo lo crea, si ya existe lo machaca.
- **x** análogo a w, pero si existe la función da fallo
- **x+**
- **c** análogo a w pero si existe no lo machaca, aunque el puntero se pone al principio
- **c+**
- **a** solo lectura. Si no existe el archivo lo crea, si ya existe empieza a escribir al final del archivo.
- **a+** lectura y escritura. Si no existe el archivo lo crea, si ya existe empieza a escribir al final del archivo.

Nos devuelve una variable a partir de la cual podemos manejar el archivo

```
<?PHP
//abre un archivo utilizando el protocolo HTTP
$archivo= fopen("datos.txt", "r")
```

```
if ( !$archivo) {  
    echo "El archivo no se puede abrir\n";  
}  
?>
```

Los modos **r**, **r+**, **w**, **w+** colocan el puntero de lectura/escritura a principio del fichero, los modos **a**, **a+** lo colocan al final.

Se pueden abrir archivos de una dirección de internet: <http://>

```
fopen("http://www.example.com/", "r");  
fopen("ftp://user:password@example.com/fichero.txt", "w");
```

fclose (indicador_archivo)

Con esta función cerramos el fichero que nos marca el indicador de archivo, devuelve TRUE si el fichero se cierra correctamente y FALSE sino se ha podido cerrar.

file_exists (fichero)

Esta función devuelve TRUE si el archivo especificado existe, y FALSE en caso contrario.

```
<?PHP  
if (file_exists("data.txt")) {  
    echo "El fichero existe";  
} else {  
    echo "El fichero NO existe";  
}  
?>
```

feof(fichero)

Devuelve TRUE si puntero de lectura/escritura se encuentra al final del fichero, y FALSE en caso contrario

fgets (indicador_archivo, longitud)

La función **fgets** nos devuelve una cadena con la longitud especifica del fichero al que apunta el indicador de archivo, o bien la línea si llegamos antes al retorno de carro o bien el EOF. Lo que ocurra antes. Si no se especifica longitud, se lee hasta fin de línea o de archivo

```
<?PHP  
$archivo= fopen("data.txt", "r");  
if ($archivo) {  
    while (!feof($archivo)) {  
        $linea = fgets($archivo, 255);  
        echo $linea;  
    }  
    fclose($archivo);  
}
```

```
}  
else  
    die("archivo no encontrado");  
?>
```

fputs (indicador_archivo, cadena)

La función **fputs** escribe una cadena en el fichero indicado. Para escribir en una archivo este debe haber sido previamente abierto. La función **fputs** devuelve TRUE si se ha escrito con éxito, en caso contrario devuelve FALSE

```
<?PHP  
$archivo = fopen("data.txt", "w");  
if ($archivo) {  
    fputs ($archivo, "HolaMundo");  
    fclose ($archivo);  
}  
?>
```

copy (origen, destino)

La función copy copia un fichero de un lugar (origen) a otro (destino), devuelve TRUE si la copia ha tenido éxito y FALSE en caso contrario.

```
<?PHP  
if (copy("data.txt", "/tmp/data.txt")) {  
    echo "El fichero ha sido copiado con éxito";  
} else {  
    echo "El fichero NO se ha podido copiar";  
}  
?>
```

10.2. Acceso a Bases de Datos

PHP ofrece interfaces para el acceso a la mayoría de las bases de datos existentes. Hay bases de datos de código abierto, como MySQL, comerciales propietarias como Oracle y además tiene librerías para acceso a datos por ODBC, lo que nos permite comunicar con todas las bases de datos posibles en sistemas Microsoft, como Access o SQL Server.

Gracias a las funciones existentes para cada sistema gestor de base de datos, podremos realizar cualquier acción con los datos que necesitemos.

Esta interacción se realiza, por un lado, a partir de las funciones que PHP nos propone para cada tipo de base de datos y, por otro utilizando SQL.

MySQL es la base de datos más extendida en combinación con PHP debido a su gratuidad, eficiencia y simplicidad.

phpMyAdmin es una aplicación web desarrollada en PHP que ofrece una interfaz para gestionar MySQL.

charset y el cotejamiento

La mayoría de los datos que se almacenan en las tablas son textos (aunque también se pueden almacenar números y datos binarios), por lo que es fundamental establecer cómo se deben codificar esos datos (ANSI, UTF-8). En la nomenclatura de MySQL se denomina charset al método de codificación y cotejamiento (*collation*) al criterio utilizado para realizar comparaciones entre textos.

Para evitar problemas podríamos sentirnos tentados a almacenar todos nuestros datos de texto en formato UTF-8, pero debemos tener en cuenta que esto supondrá multiplicar por 3 las necesidades de almacenamiento de algunos campos inútilmente.

Tipos de campos en MySQL

Los tipos de campos en MySQL pueden clasificarse fundamentalmente en tres categorías: numéricos, fechas - horas y textos.

Es importante elegir el tipo adecuado para cada dato

- Numéricos

Se pueden dividir en dos grandes grupos, los que están en coma flotante (con decimales) y los que no.

TinyInt: es un número entero con o sin signo. Con signo el rango de valores válidos va desde -128 a 127. Sin signo, el rango de valores es de 0 a 255

Bit ó Bool: un número entero que puede ser 0 ó 1

SmallInt: número entero con o sin signo. Con signo el rango de valores va desde -32768 a 32767. Sin signo, el rango de valores es de 0 a 65535.

MediumInt: número entero con o sin signo. Con signo el rango de valores va desde -8.388.608 a 8.388.607. Sin signo el rango va desde 0 a 16777215.

Integer, Int: número entero con o sin signo. Con signo el rango de valores va desde -2147483648 a 2147483647. Sin signo el rango va desde 0 a 429.4967.295

BigInt: número entero con o sin signo. Con signo el rango de valores va desde -9.223.372.036.854.775.808 a 9.223.372.036.854.775.807. Sin signo el rango va desde 0 a 18.446.744.073.709.551.615.

Float: número pequeño en coma flotante de precisión simple. Los valores válidos van desde -3.402823466E+38 a -1.175494351E-38, 0 y desde 1.175494351E-38 a 3.402823466E+38.

xReal, Double: número en coma flotante de precisión doble. Los valores permitidos van desde -1.7976931348623157E+308 a -2.2250738585072014E-308, 0 y desde 2.2250738585072014E-308 a 1.7976931348623157E+308

Decimal, Dec, Numeric: Número en coma flotante desempquetado. El número se almacena como una cadena

Tipo de Campo	Tamaño de Almacenamiento
TINYINT	1 byte
SMALLINT	2 bytes
MEDIUMINT	3 bytes
INT	4 bytes
INTEGER	4 bytes
BIGINT	8 bytes
FLOAT(X)	4 ú 8 bytes
FLOAT	4 bytes
DOUBLE	8 bytes
DOUBLE PRECISION	8 bytes

REAL	8 bytes
DECIMAL(M,D)	M+2 bytes sí D > 0, M+1 bytes sí D = 0
NUMERIC(M,D)	M+2 bytes if D > 0, M+1 bytes if D = 0

- **Fecha**

A la hora de almacenar fechas, hay que tener en cuenta que Mysql no comprueba de una manera estricta si una fecha es válida o no. Simplemente comprueba que el mes está comprendido entre 0 y 12 y que el día está comprendido entre 0 y 31.

Date: tipo fecha, almacena una fecha. El rango de valores va desde el 1 de enero del 1001 al 31 de diciembre de 9999. El formato de almacenamiento es de año-mes-día

DateTime: Combinación de fecha y hora. El rango de valores va desde el 1 de enero del 1001 a las 0 horas, 0 minutos y 0 segundos al 31 de diciembre del 9999 a las 23 horas, 59 minutos y 59 segundos. El formato de almacenamiento es de año-mes-díahoras:minutos:segundos

TimeStamp: Combinación de fecha y hora. El rango va desde el 1 de enero de 1970 al año 2037. El formato de almacenamiento depende del tamaño del campo:

Tamaño	Formato
14	AñoMesDiaHoraMinutoSegundoaaaamddhhmmss
12	AñoMesDiaHoraMinutoSegundoaamddhhmmss
8	ñoMesDiaaaaamdd
6	AñoMesDiaaamdd
4	AñoMesaamm
2	Año aa

Time: almacena una hora. El rango de horas va desde -838 horas, 59 minutos y 59 segundos a 838, 59 minutos y 59 segundos. El formato de almacenamiento es de 'HH:MM:SS'

Year: almacena un año. El rango de valores permitidos va desde el año 1901 al año 2155. El campo puede tener tamaño dos o tamaño 4 dependiendo de si queremos almacenar el año con dos o cuatro dígitos.

Tipo de Campo	Tamaño de Almacenamiento
DATE	3 bytes
DATETIME	8 bytes
TIMESTAMP	4 bytes
TIME	3 bytes
YEAR	1 byte

- **Cadena:**

Char(n): almacena una cadena de longitud fija. La cadena podrá contener desde 0 a 255 caracteres.

VarChar(n): almacena una cadena de longitud variable. La cadena podrá contener desde 0 a 255 caracteres.

Dentro de los tipos de cadena se pueden distinguir otros dos subtipos, los tipo Text y los tipo BLOB (BinaryLargeObject)

La diferencia entre un tipo y otro es el tratamiento que reciben a la hora de realizar ordenamientos y comparaciones. Mientras que el tipo text se ordena sin tener en cuenta las Mayúsculas y las minúsculas, el tipo BLOB se ordena teniéndolas en cuenta.

Los tipos BLOB se utilizan para almacenar datos binarios como pueden ser ficheros.

TinyText y TinyBlob: Columna con una longitud máxima de 255 caracteres.

Blob y Text: un texto con un máximo de 65535 caracteres.

MediumBlob y MediumText: un texto con un máximo de 16.777.215 caracteres.

LongBlob y LongText: un texto con un máximo de caracteres 4.294.967.295. Hay que tener en cuenta que debido a los protocolos de comunicación los paquetes pueden tener un máximo de 16 Mb.

Enum: campo que puede tener un único valor de una lista que se especifica. El tipo Enum acepta hasta 65535 valores distintos

Set: un campo que puede contener ninguno, uno ó varios valores de una lista. La lista puede tener un máximo de 64 valores.

Tipo de campo	Tamaño de Almacenamiento
---------------	--------------------------

CHAR(n)	n bytes
VARCHAR(n)	n +1 bytes
TINYBLOB, TINYTEXT	Longitud+1 bytes
BLOB, TEXT	Longitud +2 bytes
MEDIUMBLOB, MEDIUMTEXT	Longitud +3 bytes
LOBLOB, LONGTEXT	Longitud +4 bytes
ENUM('value1','value2',...)	1 ó dos bytes dependiendo del número de valores
SET('value1','value2',...)	1, 2, 3, 4 ó 8 bytes, dependiendo del número de valores

Conectar con una base de datos MySQL mediante PHP

Usar una base de datos MySQL mediante PHP es muy fácil gracias a las instrucciones de la librería **mysqli**. Esta extensión permite acceder a la funcionalidad proporcionada por MySQL 4.1 y posterior, la anterior librería **mysql** no implementa la POO, trabaja con procedimientos. La extensión **mysqli** ofrece una interfaz dual. Soporta el paradigma de programación procedimental y el orientado a objetos.

Los usuarios que migren desde la extensión **mysql** antigua pueden preferir la interfaz procedimental. Esta interfaz es similar a la de la extensión antigua de **mysql**. En la mayoría de los casos, los nombres de funciones difieren únicamente por el prefijo

```
<?php
$mysqli = mysqli_connect("ejemplo.com", "usuario", "contraseña", "basedatos");
if (mysqli_connect_errno($mysqli)) {
    echo "Fallo al conectar a MySQL: " . mysqli_connect_error();
}

$resultado = mysqli_query($mysqli, "SELECT 'Un mundo lleno de ' AS _msg FROM DUAL
");
$fila = mysqli_fetch_assoc($resultado);
echo $fila['_msg'];

$mysqli = new mysqli("ejemplo.com", "usuario", "contraseña", "basedatos");
if ($mysqli->connect_errno) {
    echo "Fallo al conectar a MySQL: " . $mysqli->connect_error;
}

$resultado = $mysqli-
>query("SELECT 'elecciones para complacer a todos.' AS _msg FROM DUAL");
$fila = $resultado->fetch_assoc();
echo $fila['_msg'];
?>
```

A continuación se recogen los pasos básicos del procedimiento general y las funciones implicadas:

1. Establecer la conexión con el servidor de bases de datos

Simplemente hay que crear una instancia de `mysqli` pasando los datos necesarios para la conexión: host, usuario, contraseña y nombre de la base de datos.

Modo procedimental

```
$mysqli = new mysqli(DB_HOST, DB_USER, DB_PASS, DB_DATABASE);  
if($mysqli->connect_errno > 0){  
    die("Imposible conectarse con la base de datos [" . $mysqli->connect_error . "]);  
}
```

Es posible dejar sin indicar la base de datos al crear la instancia y hacerlo posteriormente mediante `$mysqli->select_db()`. Este método devolverá un bool para verificar el posible error al seleccionar la base de datos.

POO

```
$mysqli = new mysqli(DB_HOST, DB_USER, DB_PASS);  
if(!$mysqli->select_db(DB_DATABASE)) die ($mysqli->error);
```

2. Errores

La extensión `MySQLi` nos proporciona propiedades de objeto que nos informa del último error producido en la base de datos.

- string **`$mysqli->error`**: Devuelve el último mensaje de error generado en la base de datos.
- int **`$mysqli->errno`**: Devuelve el código de error generado en la base de datos
- string **`$mysqli->connect_error`**: Devuelve el mensaje de error generado por el último intento fallido de conexión.
- int **`$mysqli->connect_errno`**: Devuelve el código de error generado por el último intento fallido de conexión.

3. Cerrar la conexión

```
mysqli_close(conexión);  
mysqli::close();
```

Es aconsejable no mezclar los dos estilos de programación, aunque sea perfectamente válido el código no es limpio.

4. Realizar una operación

Para este tipo de consultas utilizaremos el método `query()`, solo necesita como parámetro la cadena con la consulta SQL. Retorna false en caso de error, la información relativa al mismo está almacenada en las dos primeras propiedades mencionadas en el apartado 2. Si la operación se lleva a cabo con éxito retorna distintas cosas según el tipo de la misma.

5. Procesar los resultados de la consulta

La operación que llevamos a cabo puede ser una sentencia que obtiene un conjunto de resultados (`mysqli_result`) o bien una sentencia que realiza cambios en los registros de la bbdd (añadir, eliminar, modificar).

a) Si llevamos a cabo operaciones de escritura sobre la bbdd, disponemos de la variable **`mysqli::$affected_rows`** en la que se almacena el número de filas afectadas en la última operación.

b) Si la operación es una consulta, el método `query` retorna un conjunto de resultados **`mysqli_result`** que podemos manipular con sus propiedades y métodos:

- **`mysqli_result::$num_rows`** número de filas del resultado
- **`mysqli_result::$field_count`** número de campos de la consulta
- **`mysqli_result::fetch_fields`** Devuelve un array de objetos que representan los campos de un conjunto de resultados
- Para recuperar las filas de una consulta, debemos hacerlo una por una almacenándolas en un único array que vamos sobrescribiendo. Hay varios métodos para volcar una fila en un array, dependiendo de si éste es numérico o asociativo:

`mysqli_result::fetch_assoc` obtiene una fila de resultados como array asociativo

`mysqli_result::fetch_array` obtiene una fila de resultados como array asociativo o numérico, podemos pasarle un parámetro para elegir el tipo

```
$row = $result->fetch_array(MYSQLI_NUM);  
printf ("%s (%s)\n", $row[0], $row[1]);  
  
$row = $result->fetch_array(MYSQLI_ASSOC);  
printf ("%s (%s)\n", $row["Name"], $row["CountryCode"]);
```

c) `mysqli::$insert_id` Devuelve el id autogenerado que se utilizó en la última consulta. Generalmente los campos autonuméricos no se especifican al insertar registros, sino que se permite que el propio gestor de bases de datos los genere.

d) `mysqli::real_escape_string` Escapa los caracteres especiales de una cadena para usarla en una sentencia SQL, tomando en cuenta el conjunto de caracteres actual de la conexión

TRANSACCIONES

Las transacciones con la extensión MySQLi son un mecanismo que nos asegura que todas y cada de las consultas que las forman van a ser ejecutadas. En el caso de que falle alguna declaración, la transacción no se llevará a cabo.

Pero tenemos que tener cuidado con el tipo de tabla o tablas sobre las que vamos a realizar la transacción. Como sabemos los tipos más comunes de motores de almacenamiento de tablas en MySQL son MyISAM y InnoDB. Siendo el primero el que se utiliza por defecto si no se dice nada y que excluye la seguridad proporcionada por la integridad referencial. Sin embargo con este tipo no se permiten transacciones. Así que transacciones solo se pueden llevar a cabo sobre tablas InnoDB.

Fases de las transacciones

1. Iniciar transacción - Autocommit

Por defecto, MySQL se ejecuta en modo autocommit. Tener activo el modo de ejecución automática (autocommit) significa que cada consulta se tratará como una transacción individual. Lo que conlleva a que tan pronto como se ejecuta una sentencia se modifica una tabla de MySQL. Haciendo imposible la vuelta atrás (modificación permanente). Pero gracias a InnoDB y a las transacción, las consultas se puede revertir (roll back). Y para ello tenemos que desactivar el citado modo de ejecución automática. Lo que iniciará la transacción.

`boolmysqli::autocommit(bool $mode )`

```
$mysqli = new mysqli("localhost", "my_user", "my_password", "demo");  
if($mysqli->connect_errno) die($mysqli->connect_error)  
$mysqli->autocommit(false);
```

2. Consultas

```
$result = $mysqli->query($sql1);  
if ($mysqli->errno) {  
    $flag = false;
```

```
echo "Error: " . $mysqli->error . " . "; }

$result = $mysqli->query($sql2);
if ($mysqli->errno) {
    $flag = false;
    echo "Error: " . $mysqli->error . " . "; }
```

3. Finalizar o revertir transacción

Lo importante de una transacción es que el desarrollador tiene el control de cuando hacer efectiva la modificación permanente o revertir estas modificaciones ocasionadas por las consultas.

Con la primera de las siguientes funciones procederemos a hacer el commit en la base de datos. Esto quiere decir que las modificaciones serán permanentes.

Con la segunda opción revertimos la transacción actual y por lo tanto ninguna de las modificaciones de las consultas se hacen efectivas.

```
boolmysqli::commit ( void )
boolmysqli::rollback ( void )

if ($flag) { //si no ha habido error en las consultas
    $mysqli->commit();
    echo "Consultas ejecutadas correctamente";
} else {
    $mysqli->rollback();
    echo "Todas las consultas han sido revertidas";
}
```

```
1 $conex->autocommit(false);
2
3 try {
4     $conex->query("INSERT INTO users (name) VALUES ('marcus')");
5     $conex->query("UPDATE users SET name = 'jane' WHERE 'id' = 39 ");
6     $conex->commit();
7 } catch (Exception $e) {
8     $conex->rollback();
9     echo 'Something fails: ', $e->getMessage(), "\n";
10 }
```

Gestión de privilegios en MySQL

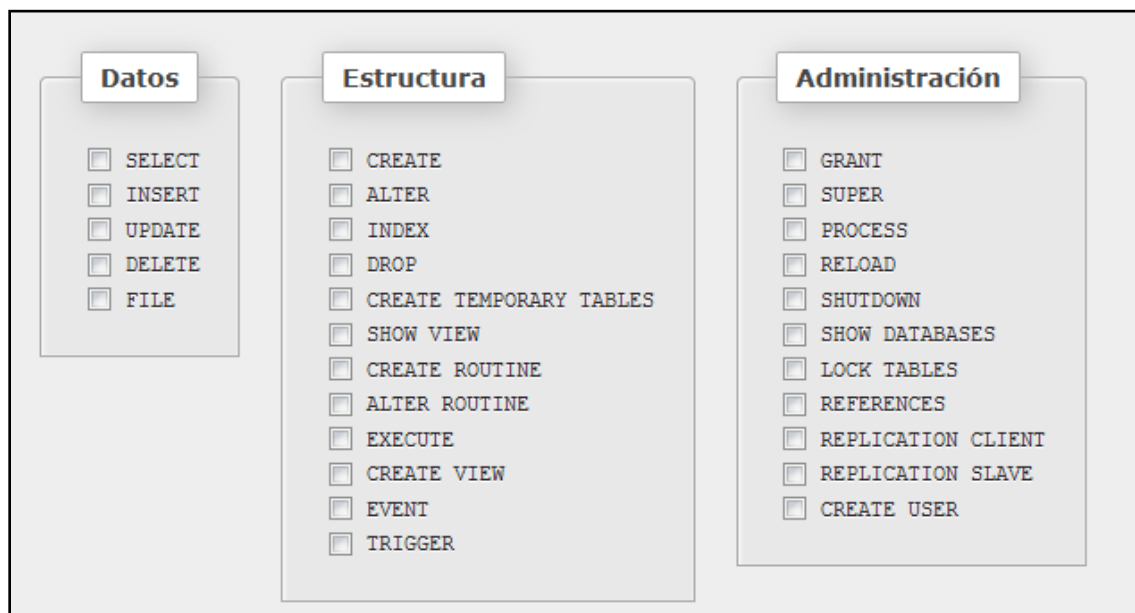
El control de acceso al gestor de bases de datos relacionales MySQL se basa en el uso de usuarios y credenciales, a los que se les pueden conceder privilegios a nivel global, a nivel de base de datos, a nivel de tabla y a nivel de columna.

Estos usuarios se almacenan en la tabla user de la base de datos mysql.

Una cuestión que suele causar confusión es la presencia del campo servidor (*host*) al crear un usuario nuevo en MySQL. Este campo puede rellenarse con el comodín %, que indica que un usuario concreto puede acceder desde cualquier servidor (incluido localhost), pero como por defecto MySQL incluye una definición de usuario más específica para las conexiones desde localhost (usuario: cualquiera, host: localhost) con unos privilegios muy limitados, se aplica esta definición y ese usuario concreto no puede acceder desde localhost. Para resolver esto podemos crear otra definición de ese usuario concreto eligiendo expresamente como servidor localhost (o borrar la definición más específica de MySQL).

Los privilegios de MySQL se clasifican en 3 grupos:

- Privilegios de administración.
- Privilegios de estructura.
- Privilegios de datos.



Los privilegios de datos están directamente relacionados con las instrucciones correspondientes de SQL (SELECT, INSERT, UPDATE y DELETE) explicadas anteriormente.

Los privilegios de estructura más importantes son CREATE, ALTER y DROP, que igualmente están relacionados con las correspondientes instrucciones SQL explicadas anteriormente. No obstante, el otorgamiento de estos privilegios no implica directamente que el usuario vaya a poder realizar la operación correspondiente; por ejemplo, para poder hacer ALTER TABLE, el usuario deberá tener CREATE e INSERT además del ALTER; y para renombrar una tabla necesitará ALTER, DROP y CREATE.

Por último, el privilegio de administración más importante es GRANT, que nos permite otorgar o retirar (mediante la instrucción SQL REVOKE) a otros usuarios los permisos que nosotros poseemos.

Una limitación importante del sistema de privilegios de MySQL es que no nos permite otorgar los privilegios DROP y CREATE a nivel de tabla; si un usuario dispone de alguno de estos privilegios a nivel de tabla también podrá aplicarlo a nivel de la base de datos a la que pertenece la tabla, es decir podrá borrar o crear la base de datos (aunque aparentemente no pueda a través de la interfaz de phpMyAdmin, sí podría hacerlo a través de la consola).

Podemos elegir una base de datos concreta sobre la que configurar sus privilegios. También podemos elegir una tabla concreta sobre la que configurar sus privilegios. Si optamos por esta segunda posibilidad accederemos a un último formulario en el que podremos establecer los privilegios a nivel de tabla y de campo.

SELECT	INSERT	UPDATE	REFERENCES	
nombre clave	nombre clave	nombre clave	nombre clave	☐ DELETE
				☐ CREATE
				☐ DROP
				☐ GRANT
				☐ INDEX
				☐ ALTER
				☐ CREATE_VIEW
				☐ SHOW_VIEW
				☐ TRIGGER
O ☐ Ninguno	O ☐ Ninguno	O ☐ Ninguno	O ☐ Ninguno	

Continuar

Seguridad en la Base de Datos

La seguridad en las bases de datos es un asunto crucial. Deberíamos aplicar siempre los siguientes principios generales:

- Validar los datos para asegurarnos de que antes de salir de PHP son del tipo que se espera en MySQL.
- Sanear los datos para asegurarnos de que antes de salir de PHP tienen el formato que espera MySQL (por ejemplo, las fechas en formato YYYY-MM-DD).
- Limitar la longitud de los datos. No permita que se introduzcan textos demasiado largos en su base de datos si no es estrictamente necesario; podrían ser códigos maliciosos.
- Escapar todas las cadenas de caracteres con `mysql_real_escape_string`.

Inyección SQL: se inserta código SQL dentro del código ya programado con el fin de alterar el funcionamiento del programa y lograr que se ejecute el código invasor

```
consulta = "SELECT * FROM usuarios WHERE nombre='$nUsuario';"
```

Si el operador escribe un nombre la aplicación generaría una sentencia correcta, en donde se seleccionarían todos los registros con dicho nombre en la base de datos:

```
SELECT * FROM usuarios WHERE nombre = 'xxxx';
```

Pero si un operador malintencionado escribe como nombre de usuario a consultar:

```
"Pepe; SELECT * FROM datos WHERE nombre LIKE '%"
```

se generaría la siguiente consulta SQL

```
SELECT * FROM usuarios WHERE nombre = 'Pepe';SELECT * FROM datos  
WHERE nombre LIKE '%";
```

En la base de datos se ejecutaría la consulta en el orden dado, se seleccionarían todos los registros con el nombre 'Pepe', y se seleccionaría toda la tabla "datos", que no debería estar disponible para los usuarios web comunes.

```
$consulta = "SELECT titulo FROM libros WHERE codigo = $codigo";
```

Si el valor es '23; DROP TABLE users' la consulta es:

```
SELECT titulo FROM libros WHERE codigo = 23; DROP TABLE users
```

```
$consulta = "SELECT id FROM usuarios WHERE username = '$username'  
AND password = '$password'";
```

Si `$username= juan'#`

```
SELECT id FROM usuarios WHERE username = 'juan'#' AND password = ''
```

es un comentario de línea en MySQL y provoca que se ignore todo lo que viene tras él en la línea

```
mysqli_real_escape_string($nombre_usuario)
```

- Usar un usuario para la conexión de PHP que posea los privilegios mínimos para realizar las operaciones que necesitamos.
- No almacenar datos que realmente no sean necesarios. Por ejemplo, no almacene un número de tarjeta de crédito si no va a usarlo en el futuro.
- Configurar mysql para que la bbdd sólo pueda ser accedida desde ips concretas, de esta manera se puede restringir el acceso sólo desde el servidor dónde está la aplicación.
- Si sólo necesitamos saber si un dato se ha escrito correctamente (por ejemplo una contraseña), pero no necesitamos saber exactamente cuál es ese dato, podemos almacenarlo hasheado en la base de datos (por ejemplo con sha512).

Si en nuestra página web tenemos un sistema de usuarios y queremos proteger las contraseñas para prevenir posibles vulnerabilidades en nuestro servidor, es una medida eficaz encriptar las contraseñas, de manera que si alguien puede acceder a ellas no pueda ver la contraseña si no su encriptación.

Pero si no usamos una transmisión segura cuando el usuario envía los datos al servidor, la contraseña es enviada sin encriptar, en ese momento puede ser capturada por un tercero. Para evitar esto, podemos encriptar la clave en el ordenador del cliente usando JavaScript.

- Si hay que almacenar datos sensibles, y debemos conocerlos podemos encriptarlos previamente en PHP mediante la librería Mcrypt (<http://php.net/manual/es/book.mcrypt.php>).